Building Resilient Distributed Systems

PDC Assignment 2 | Student ID: bscs23214 | Parallel and Distributed Computing

Part 1: Analysis of the Naive Architecture

Problem 1 — Lost Update (Synchronisation)

The root cause is that the server does a blind save — it runs an UPDATE on the database without first checking whether someone else already changed the document. Here is what happens: both Alice and Bob open the same document and see version 1. Alice edits and saves first. The database now has her changes. Then Bob saves his version, which he edited from the old copy he opened. The server just overwrites whatever is there. Alice's work is gone, and nobody gets an error message. This is called the **Lost Update anomaly** — data is destroyed silently.

Problem 2 — Dropped Webhook (Coordination)

When a user cancels their subscription, Clerk fires an HTTP webhook to our server. If the network has a hiccup at that exact moment, or the server is restarting, the webhook never arrives. Clerk will retry a few times, but after that it gives up. The result: the cancellation is never recorded, and the user keeps premium access forever. There is also no duplicate check, so if the same webhook arrives twice (which Clerk can do), the cancellation gets applied twice and the account breaks.

Problem 3 — Synchronous LLM Call (Fault Tolerance)

The server calls the external AI service and then just sits and waits — doing nothing else until it gets a reply. If the AI is down or slow, that wait can be 60 seconds. FastAPI can only handle so many requests at once. If several users hit the AI endpoint at the same time, all those server slots get stuck waiting. The entire app then stops responding for everyone, not just the users asking the AI. One slow external service becomes a **single point of failure** for the whole system.

Part 2: Architectural Design

Fix 1 — Optimistic Locking with Version Numbers (Sync)

Every document gets a version number that starts at 1 and goes up by 1 on each save. When a user loads a document, the version comes along with it. When they save, they must include the version they last saw. The database update is guarded like this:

```
UPDATE documents SET body=?, version=version+1 WHERE id=? AND version=?
```

If the version still matches, the save works and the version increases. If someone else saved first and bumped the version, the WHERE clause finds zero rows. The server returns **HTTP 409 Conflict**. The user is told to reload, merge their changes with the new version, and try again. Nothing is ever lost silently.

Trade-off: This works great when conflicts are rare, which is normal for document editing. If many users constantly edit the same document, they will keep getting 409 errors and retrying, which adds latency. For very high-conflict cases, pessimistic locking (row-level locks) avoids retries but blocks other readers too.

UML Sequence Diagram — Optimistic Locking (Two Concurrent Users)

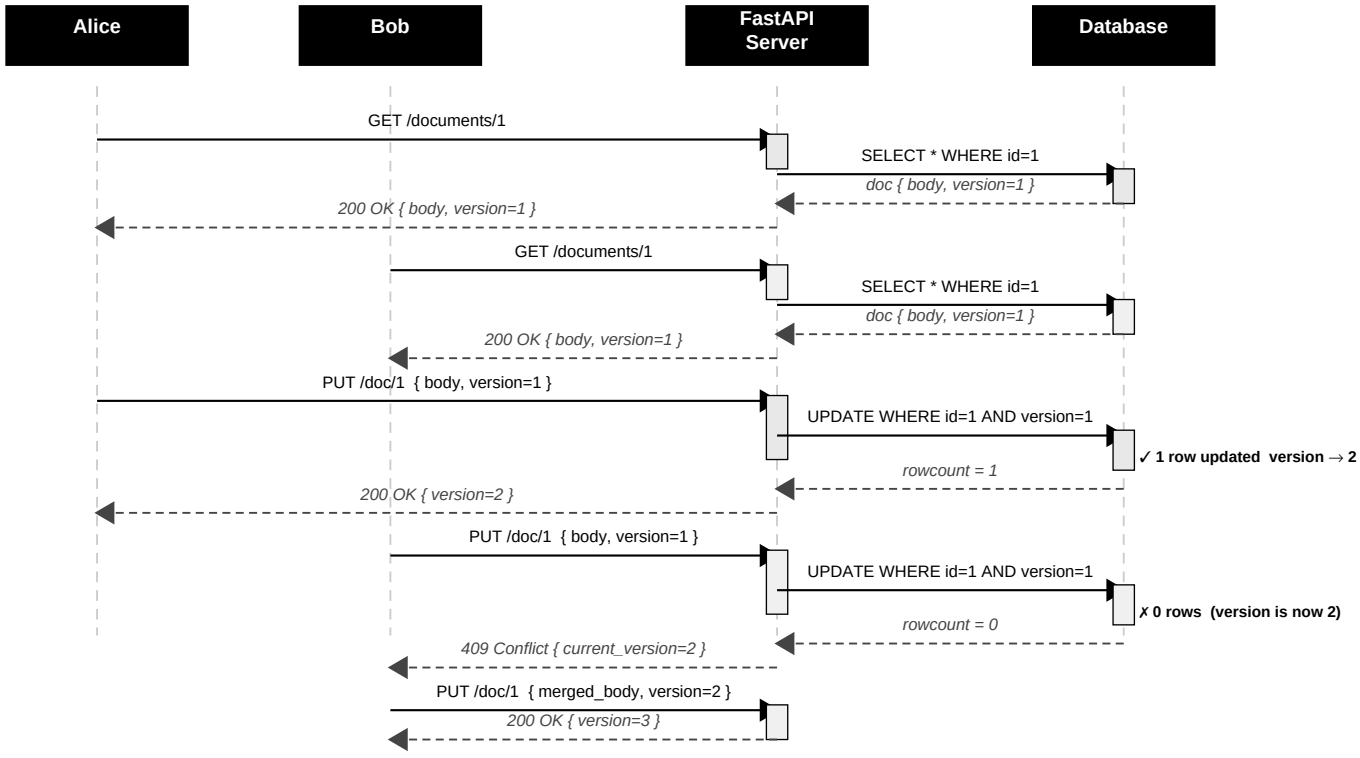

Figure 1: Alice and Bob both read version 1. Alice saves first (version becomes 2). Bob gets HTTP 409 Conflict because his version is stale. He re-fetches, merges his changes, and saves successfully with version 2 (version becomes 3).

Fix 2 — Idempotent Webhook Handler with Dead-Letter Queue (Coordination)

Three things work together so no cancellation is ever lost or applied twice:

- **Idempotency key:** Clerk puts a unique ID on every webhook. When it arrives, we first check if we already processed that ID. If yes, we return 200 and do nothing. If no, we process it and save the ID in the same database transaction.
- **Transactional inbox:** We save the raw webhook to a pending table first, then a background worker applies it. Even if the server crashes mid-way, the webhook is still in the table and will be picked up on the next run.
- **Dead-letter queue (DLQ):** If a webhook fails after several retries, it moves to an error table and the team gets an alert. Someone can look at it and replay it manually. No cancellation ever disappears silently.

Fix 3 — Circuit Breaker with Fallback for LLM API (Fault Tolerance)

A Circuit Breaker wraps every call to the AI service. It works like the circuit breakers in your home — when something goes wrong, it trips open so the problem does not spread. It has three states:

- **CLOSED (normal):** Requests go through to the AI. After 3 failures in a row, it trips to OPEN.
- **OPEN (tripped):** No requests reach the AI at all. A ready-made fallback reply is sent instantly — the app stays fast and responsive. After 30 seconds, it moves to HALF-OPEN.
- **HALF-OPEN (testing):** One test request goes through. If it works, the breaker closes again. If it fails, it goes back to OPEN.

Every AI call also has a hard timeout of 10 seconds. This makes sure no single request can ever block the server for the full 60 seconds.

CAP Theorem Trade-offs

The CAP theorem says a distributed system can only fully guarantee two of three things: **Consistency** (all users see the same data), **Availability** (the system always responds), and **Partition Tolerance** (it keeps working even if the network has problems). Since network issues always happen in the real world, we have to choose between C and A for each problem. Here is what we chose and why:

Problem	Choice	Reason
Lost Update (Version Lock)	Consistency over Availability	A 409 temporarily stops one user, but no data is ever destroyed. Getting the data right matters more than always accepting saves instantly.
Dropped Webhook (Inbox + DLQ)	Consistency over Availability	We slow down webhook processing to make sure it happens exactly once. Applying a cancellation twice — or not at all — would be worse.
LLM Timeout (Circuit Breaker)	Availability over Consistency	We return a fallback message instead of hanging the app. The answer may not be the latest AI output, but the app keeps working for everyone.